

API LISTEN/NOTIFY

Краткий обзор

СУБД PostgreSQL имеет встроенную поддержку шины сообщений, реализованную в виде нестандартных SQL команд LISTEN, UNLISTEN и NOTIFY, а также функции pg_notify.

Сообщения посылаются в канал, на который можно подписываться. Название канала — произвольный идентификатор SQL, обычно он совпадает с названием таблицы, с которой связано какое-либо событие. Тело сообщения опционально, является произвольной строкой с длиной не более 8000 символов. Сообщение гарантированно доставляется всем подписчикам, при этом учитываются границы транзакций для соблюдения принципов ACID. Время жизни подписки — от команды LISTEN до команды UNLISTEN или завершения соединения с БД.

Для подробной информации можно обратиться к официальной документации PostgreSQL: [NOTIFY](#), [LISTEN](#) и [UNLISTEN](#)

Поддержка LISTEN/NOTIFY в ядре платформы

Для отправки сообщения нужно выполнить SQL команду NOTIFY или функцию pg_notify с помощью метода [IStatement::Exec](#) или [IStatement::ExecPrepared](#).

Для подписки на канал нужно вызвать метод [IDatabase::Listen](#) или [IConnection::Listen](#) с именем канала. Метод Listen возвращает объект типа [INotifyListener](#), предоставляющий механизм ожидания и извлечения полученных сообщений. В рамках одного соединения с БД поддерживается только один подписчик на канал, последующие вызовы Listen с тем же именем канала будут завершаться выбросом исключения, пока соответствующий экземпляр [INotifyListener](#) не будет освобождён или отписан. Освобождение [INotifyListener](#) автоматически прекращает подписку.

По вызову Listen драйвер начинает асинхронный приём сообщений от БД. Полученное сообщение ставится в очередь экземпляра [INotifyListener](#), из которой его можно забрать с помощью метода [INotifyListener::Next](#). Количество сообщений в локальной очереди ограничено и не может превышать MAX_PENDING_NOTIFIES (50 сообщений). При превышении этого количества самое старое сообщение отбрасывается.

Метод [INotifyListener::Next](#) возвращает std::unique_ptr< INotify >. Класс [INotify](#) имеет методы:

- [ChannelName\(\)](#) — возвращает имя канала, из которого пришло сообщение;
- [Message\(\)](#) — возвращает тело сообщения, которое может быть пустым;
- [ProcessId\(\)](#) — возвращает номер серверного процесса БД, его можно использовать для проверки, было ли сообщение отправлено из текущего сеанса работы с БД.

Поддержка в Python

В основном аналогично C++. Можно пользоваться функцией [SqlQuery](#) для выполнения команды NOTIFY.

Методы класса INotify называются немного по-другому:

- Channel() — возвращает имя канала, из которого пришло сообщение;
- Message() — возвращает тело сообщения, которое может быть пустым;
- Pid() — возвращает номер серверного процесса БД.

Примеры

Отправка сообщения

Для отправки сообщения можно использовать SQL команду NOTIFY или функцию pg_notify. Функция удобнее, если название канала не фиксировано, поскольку команда NOTIFY в качестве первого параметра ожидает идентификатор, а не строку.

Пример на C++.

```
1 auto stmt = db->CreateStatement();
```

```

2 stmt->SetParam( 1, channel_name );
3 stmt->SetParam( 2, notify_message );
4 stmt->Exec( L"select pg_notify( $1, $2 )" );

```

Пример на Python.

```
SqlQuery('NOTIFY my_channel, $1', notify_message)
```

Получение сообщения

Метод [INotifyListener::Next](#) может ожидать поступления сообщения ограниченное время или бесконечно долго.

Пример на C++.

```

1 // Подписываемся на сообщения.
2 auto listener = dbi::GetConnectedDataDB()->Listen( channel_name );
3
4 // Ждем сообщение бесконечно долго.
5 auto notify = listener->Next();
6 LogMsg( L"Получили сообщение: ", notify->Message() );
7 // Ждем сообщение в течение указанного периода времени.
8 if( auto notify = listener->Next( Milliseconds{ 100 } ) )
9     LogMsg( L"Получили сообщение: ", notify->Message() );
10 // Ждем сообщение до указанного момента времени.
11 if( auto notify = listener->Next( DateTime::Now() + Milliseconds{ 100 }
12     ) )
13     LogMsg( L"Получили сообщение: ", notify->Message() );

```

Пример на Python.

```

1 # Подписываемся на сообщения с гарантией отписки по выходу из блока
  with.
2 with sbis.GetConnectedDatabase().Listen(channel_name) as listener:
3     # Ждем сообщение бесконечно долго.
4     notify = listener.Next()
5     sbis.LogMsg('Получили сообщение: ' + notify.Message())
6     # Ждем сообщение в течение указанного периода времени.
7     notify = listener.Next(timedelta(milliseconds=100))
8     if notify:
9         sbis.LogMsg('Получили сообщение: ' + notify.Message())
10    # Ждем сообщение до указанного момента времени.
11    notify = listener.Next(datetime.now() +
12        timedelta(milliseconds=100))
13    if notify:
14        sbis.LogMsg('Получили сообщение: ' + notify.Message())

```